



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۶ آبان

فصل سوم: لایه *Transport* اسلاید ۱ تا ۳۱ [۱]

لایه انتقال یک ارتباط منطقی و مجازی بین پروسس‌های *Host* های مبدا و مقصد ایجاد میکند. پروتکل‌های انتقال فقط در *Host* های پایانی اجرا میشود نه در *core* شبکه. در واقع لایه انتقال، در مبدا *message* ها را از لایه کاربرد گرفته و میشکند و تبدیل به *segment* میکند و به لایه شبکه میدهد. در مقصد هم، آن *segment* ها را از شبکه گرفته و بعد سر هم کرده و به *message* تبدیل میکند و به لایه کاربرد میدهد. لایه شبکه ارتباط منطقی بین میزبان‌هاست اما لایه انتقال ارتباط منطقی بین پروسس‌های *host* های مبدا و مقصد برقرار میکند. سرویس لایه انتقال از سرویس‌های لایه شبکه استفاده میکند و میتوان با ارتقای سرویس لایه شبکه امکانات اضافی‌تری در سرویس لایه انتقال داده باشیم. دو پروتکل لایه انتقال *TCP* و *UDP* هستند. *UDP* بدون ترتیب و نامطمئن و بدون هیچ اضافاتی مثل همان *Best effort* برای *IP* در لایه *network* است. فرستنده داده‌های سوکت‌های مختلفش را میگیرد، سرایند لایه انتقال را اضافه میکند و میفرستد سرایند هایی که به پیام‌ها اضافه می‌شود، در گیرنده *demultiplex* می‌شود تا به سوکت و پروسس مورد نظر در لایه کاربرد مقصد برسند.

چطور *demultiplexing* کار میکند؟

دیتاگرام‌هایی که مقصد میگیرد، دارای آدرس *IP* مبدا و مقصد هستند. همچنین یک بخش *segment* مربوط به لایه انتقال دارند که دارای پورت مبدا و مقصد است. میزبان به کمک آدرس *IP* و شماره پورت میتواند بسته‌ها را تمیز دهد و به سوکت مورد نظر بدهد.

UDP در Demultiplex

این امکان وجود دارد که پورت مبدا را در UDP مشخص کنیم اما فقط از روی آدرس IP مقصد و پورت مقصد این کار *demultiplex* را انجام می‌دهد.

TCP در Demultiplex

برخلاف UDP به ۴ تا چیز نیاز دارد تا *demultiplex* کند و به سوکت مناسب بفرستد. که این ۴ تا، آدرس IP و پورت مبدا و مقصد هستند.

هر سوکت با این ۴ تایی مشخص میشود.

سرور های وب سوکت متمایز برای هر مشتری که متصل می‌شود در نظر می‌گیرند.

در *non – persistent HTTP* برای هر درخواست یک سوکت متمایز دارد.

مثلا پورت مقصد با اینکه ۸۰ است و آدرس مقصد یکسان هستند اما چون مشتری ها در پورت مبدا و آدرس مبدا با هم فرق دارند، در مقصد به سوکت های مختلفی هدایت می‌شوند.

UDP

همان سرویس *best effort* بدون اضافات است. پس ممکن است سگمنت ها *Loss* شوند یا خارج ترتیب برسند. بدون اتصال و *handshake* است.

هر سگمنت به طور مستقل از بقیه اداره می‌شود.

برای کاربرد های *streaming* خوب هستند، چون کنترل ازدحام ندارد و نرخ خود را کم نمیکند. همچنین در کاربرد های *streaming* میتوان کمی *Loss* را تحمل کرد اما حداقل *bandwidth* می‌خواهد و حساس به نرخ هستند.

برای پروتکل های *DNS* و *SNMP* (که پروتکلی برای این است که در شرایط ازدحام بتواند عملیاتی باشد و چون *UDP* کنترل ازدحام ندارد، برای آن خوب است.) نیز به کار میرود

برای داشتن *reliability* در *UDP*:

در *HTTP 3* اگر بخواهیم کاربرد اطمینان پذیری داشته باشد، سرویس مربوط به *reliability* آن را در لایه کاربرد اضافه میکنیم و از *UDP* استفاده می‌کنیم.

سرایند هر سگمنت در *UDP*:

دو تا ۳۲ بیت در مجموع ۸ بایت. ۴ تا فیلد ۱۶ بیتی یک پورت مبدا و یک پورت مقصد یک فیلد مربوط به طول کل سگمنت(با در نظر گرفتن طول خود سرایند) و فیلد *checksum* برای تشخیص خطا

چرا از *UDP* استفاده میکنیم؟

- سریع است، چون بدون اتصال اولیه و یک RTT کمتر می‌شود است.
- ساده است و نیاز به نگهداری $state$ در مبدا و مقصد نیست.
- کنترل ازدحام ندارد و با هر نرخى بخواهد و در هر شرایطی، سگمنت را ارسال میکند.
- اندازه خود سرایند آن هم کوچک ترست و $overhead$ کمتری در هر سگمنت دارد.

checksum

گیرنده میاد $content$ و محتوای اون $segment$ رو هب صورت ۱۶ بیت ۱۶ بیت در نظر میگیره و بعد اون ها رو به صورت $1's complement$ با هم جمع میکنه. (احتمالا خود فیلد $checksum$ الان صفره) و نتیجه رو توی فیلد $checksum$ میذاره. گیرنده هم دقیقا با پیام همینکارو میکنه (به غیر از فیلد $checksum$ ش احتمالا). یعنی اونو مثل محاسبه فرستنده صفر میگیره)

اگر $checksum$ ها مخالف باشند، یعنی در سگمنت در طول راه خطا رخ داده. اگر یکی باشند یعنی $no error detection$ نه اینکه خطا نداریم، چون ممکن است خطا رخ داده باشد اما به نحوی باشد که $checksum$ ها یکی شده اند. پروتکل انتقال داده مطمئن:

این سرویس انتقال داده مطمئن در لایه انتقال پیاده سازی و ارائه می‌شود می‌شود، به نوعی یک کانال مطمئن بین مبدا و مقصد برای انتقال پیام درست میکند. پروتکل های سرویس انتقال مطمئن هم در فرستنده و هم گیرنده وجود دارند. هر چه قدر لایه شبکه نامطمئن تر باشد، پیچیدگی طراحی پروتکل لایه انتقال مطمئن، بیشتر است. و باید معطوف به آن طراحی شود.

پروتکل $rdt = reliable data transfer$

این پروتکل از ۴ تا $subroutine$ تشکیل شده است.

(۱) $rdt_send()$: کاربرد فرستنده، سابروتین $rdt_send()$ را فرا میخواند به پروتکل انتقال فرستنده میدهد.

(۲) $udt_send()$: فرستنده که rdt سمت فرستنده را پیاده سازی کرده سرایند را اضافه میکند و بسته را میسازد و با سابروتین $udt_send()$ میفرستند و لایه شبکه آن را از کانال نامطمئن انتقال می‌دهد.

(۳) $rdt_rcv()$: مقصد یا سابروتین $rdt_rcv()$ بسته را میگیرد.

(۴) $deliver_data()$: بعد که سگمنت را گرفت، لایه انتقال طبق پروتکل rdt سرایند مربوط به آن را جدا کرده و حاصل را با این سابروتین $deliver_data()$ به لایه بالاتر، یعنی لایه کاربرد تحویل می‌دهد

ارتباط کاربرد فرستنده و گیرنده دو طرفه است.

سناریو های مختلف *rdt*

در اسلاید ۲۳، انتقال داده یک طرفه و انتقال سیگنال های کنترلی دو طرفه فرض شده است. از $FSM = finite\ state\ machines$ برای مدل سازی این پروتکل های *rdt* استفاده می‌کنیم. هر دایره یک حالت است. صورت کسر هر یال، پیشامد باعث تغییر حالت و مخرج کسر در یال، عملی که در طی آن تغییر حالت رخ می‌دهد را میگوید. (بنظرم اسلاید های ۲۳ تا ۳۱ را ببین تا درک بهتری پیدا کنی)

rdt 1

نه خطای بیت داریم و نه اتلاف بسته

برای هر سمت مبدا و مقصد، یک *FSM* داریم. هر دو هم یک حالت دارند. مبدا در حالت *wait for a call from above* است در مبدا اگر $rdt - send()$ فراخوانده شود، داده را می‌شکنیم به بسته های کوچک تر و سرایند سگمنت را بهش اضافه میکنیم با $udt - send()$ میفرستیم. از اون ور در *FSM* مقصد (که منتظر دریافت پیام از لایه های پایین است)، با $rdt - rcv()$ دریافت میکند و سرایندها را جدا میکند و پیام اولیه را دوباره از دل بسته ها میسازد و به سابروتین $deliver - data()$ میدهد تا به کاربرد برسد.

rdt 2

ممکن است کانال زیرین برعکس قبلی بعضی بیت ها را معکوس میکند و *bit error* داریم. در اینجا از *checksum* استفاده میکنیم تا گیرنده بفهمد بیت ها دچار خطا شدند. هر بسته باید توسط گیرنده تصدیق شود. اگر درست بود، به فرستنده *ACK* مبدد و اگر خراب بود از *NAK* به فرستنده میفرستد.

اگر فرستنده *NAK* بگیرد، همان بسته را دوباره *retransmit* میکند.

پس اینجا در *rdt 2* هم تشخیص خطا داریم هم *feedback* فرستادن.

درواقع *FSM* مبدا دو حالت دارد، حالت دیفالت که مثل قبل برای است و منتظر *call* شدن و فرستادن بسته. حالت دوم، *wait for ACK or NAK* است که منتظر میماند تا از گیرنده تصدیق درستی بسته را بگیرد. (گیرنده با همان $udt - send()$ پیام *ACK* و *NAK* را میفرستد.)

در *rdt 2* فرستنده از حالت گیرنده خبر ندارد و تنها وقتی مطلع می شود که یک پیام بگیرد. برای همین به پروتکل نیاز داریم.

ممکن است پیام *ACK* یا *NAK* خراب شوند و فرستنده پیام گیرنده را نمیتواند *decode* کند و پیام آن و استیت آن را نمیفهمد. پس ممکن است پیام را دوباره ارسال کند و برای همین ممکن است بسته های تکراری در گیرنده داشته باشیم.

sequence number

برای تشخیص جدید یا تکراری بودن بسته، در سرایند بسته یک *sequence number* اضافه می‌شود. فرستنده بسته را باز ارسال میکند و گیرنده از روی *sequence number* تشخیص میدهد و اگر تکراری باشد آن را حذف میکند.

stop and wait

پروتکل آن از نوع *stop and wait* است. یعنی فرستنده یک بسته میفرستد و منتظر جواب گیرنده میماند.

مراجع

[1] “ch3-v8-slide-1-to-31,” <https://drive.google.com/drive/folders/>, 2022.